

🎙️ Rapport d'Activité Final : Plateforme de Simulation Bancaire BMCI & Agent Vocal Temps Réel (LiveKit)

Ce rapport d'activité retrace de manière exhaustive et structurée l'ensemble des travaux de recherche, de développement et d'expérimentation menés pour concevoir une plateforme de simulation bancaire interactive et un agent vocal temps réel pour la **BMCI**.

1. Objectifs Généraux & Contexte

Le projet a évolué d'une simple étude de modèles à la création d'une **plateforme de simulation bancaire pour la BMCI**. L'objectif est d'entraîner et d'évaluer des modèles d'intelligence artificielle pour qu'ils incarnent le rôle strict d'un client bancaire (ex: **M. Orens**, client pressé et mécontent) dans des conversations de formation avec un conseiller (l'utilisateur apprenant).

2. Phase 1 : Plateforme de Simulation BMCI & Fine-Tuning de LLMs Locaux

A. Création du Dataset Bancaire Français

Pour adapter les modèles au contexte métier de la BMCI et leur apprendre à se comporter comme des clients réalistes, nous avons constitué un **jeu de données (dataset) fictif bancaire en français**. Ce dataset couvre 9 scénarios clés de la vie courante en agence :

1. Carte bancaire bloquée.
2. Virement en retard.
3. Application mobile bloquée.
4. Frais bancaires injustifiés.
5. Demandes de crédit.
6. Ouverture de compte.
7. Encaissement / Problèmes de chèque.
8. Augmentation de plafond de carte.
9. Contestation d'opération frauduleuse.

B. Fine-Tuning LoRA de Modèles Légers

Pour exécuter l'agent en local sur des machines standard, nous avons testé le fine-tuning léger via la méthode **LoRA** (Low-Rank Adaptation) sur plusieurs architectures de petits modèles :

- **Modèles testés** : *Qwen (0.5B/1.5B)*, *SmolLM2*, *TinyLlama*, *BloomZ* et *CroissantLLM*.
- **Résultat de l'entraînement** : Le modèle le plus stable et exploitable localement à ce jour est **Qwen2.5-0.5B-BMCI-Client-Finetune-1**. Il démontre une bonne assimilation des cas d'usage bancaires malgré sa taille réduite.

C. Adaptation de l'Application Streamlit

L'interface de démonstration sous Streamlit a été entièrement refondue pour intégrer :

- **Gestion locale** : Chargement direct des modèles français et des checkpoints fine-tunés locaux.
 - **Flexibilité des données** : Prise en charge automatique des formats de datasets locaux (.json, .jsonl, .csv).
 - **Nouvel onglet "Chat Client BMCI"** : Interface de dialogue direct pour tester le comportement de jeu de rôle.
 - **Garde-fous intégrés (Guardrails)** : Filtres automatiques bloquant les réponses où le modèle oublie son rôle et commence à se prendre pour le conseiller bancaire.
 - **Sécurisation de la mémoire** : Blocage automatique des modèles trop lourds (ex : *Mistral 7B* en local) pour éviter les saturations de RAM/VRAM et les crashes de l'application.
-

3. Phase 2 : Évaluation des Modèles & Sécurité (Framework & Guardrails)

Nous avons mis en place un framework d'évaluation automatisé rigoureux, inspiré des meilleures pratiques de la recherche :

- **Scénarios multi-modèles (approche Promptfoo)** : Comparaison simultanée de plusieurs modèles face aux mêmes invites (prompts).
 - **Cas Red-Team (Tests de Robustesse)** : Attaques verbales simulées pour tenter de faire sortir le modèle de son rôle de client.
 - **Scoring applicatif (approche DeepEval / G-Eval)** : Évaluation automatique de la qualité des réponses à l'aide de critères LLM-as-a-judge.
 - **Mesure de fidélité (proxy RAGAS)** : Analyse de la cohérence de la réponse vis-à-vis du contexte de la conversation.
 - **Métriques clés mesurées** :
 - Respect du rôle (L'IA reste-t-elle le client ?).
 - Pertinence de la réponse métier.
 - Sécurité (Non-divulgence d'informations sensibles).
 - Taux d'hallucination et concision.
 - Déclarations non supportées (claims non fondés).
-

4. Phase 3 : Intégration Voix Temps Réel (LiveKit & WebRTC)

Pour transformer cette simulation en expérience vocale en direct, nous avons connecté les modèles à un flux audio temps réel via **LiveKit**.

- **STT (Reconnaissance Vocale)** : Utilisation de **Cohere Transcribe v2**. Très robuste pour transcrire fidèlement le français malgré le bruit de fond d'une agence.
- **LLM (Cerveau)** : **Mistral (mistral-small-latest)** via l'adaptateur OpenAI pour une intelligence fluide.
- **TTS (Synthèse Vocale)** : **Mistral Voxtral** avec la voix nativement irritée de Marie (`fr_marie_angry`) pour incarner la frustration de M. Orens.

- **Ajustements de fluidité :**

- **Nettoyage textuel automatique** : Un filtre de remplacement Regex retire les majuscules d'insistance (MON → mon) et les didascalies entre crochets ([gasp]) pour éviter que la synthèse vocale ne les épelle à haute voix, tout en les gardant affichés dans la console.
 - **Optimisation VAD (min_delay=0.8)** : Une attente de silence de 0,8s configurée pour s'assurer que la transcription complète soit reçue avant que l'IA ne prenne la parole, évitant de couper l'utilisateur.
-

5. Difficultés Pratiques & Matérielles Surmontées

□ A. Respect du Rôle (Le modèle se prend pour le conseiller)

- *Difficulté* : Les petits modèles (0.5B à 3B) ont tendance à oublier qu'ils jouent le client et se mettent à répondre à la place du conseiller.
- *Solution* : Renforcement des prompts système avec des instructions de formatage strictes, ajout d'exemples *few-shot* d'échanges types, et intégration de filtres de blocage post-génération.

📄 B. Dérives Contextuelles lors du Fine-Tuning

- *Difficulté* : Bien que le modèle fine-tuné comprenne son rôle de client, il dérive parfois du contexte précis (ex: parler de carte bloquée alors que le scénario traite d'un virement en retard).
- *Solution* : Nécessité d'élargir le dataset de fine-tuning avec plus d'exemples ciblés et des cas d'auto-correction.

🖥️ C. Limitations Matérielles en Local (GPU Intel vs. Nvidia)

- *Difficulté* : Impossibilité d'entraîner les modèles en local sous Windows (le processeur graphique Intel intégré n'étant pas compatible avec CUDA). Les modèles lourds (Mistral 7B) faisaient crasher la mémoire Streamlit.
- *Solution* : Déportation de toute la phase d'entraînement sur des environnements cloud gratuits (**Google Colab** et **Kaggle**) et blocage préventif des modèles de plus de 3B paramètres en local.

💾 D. Problèmes d'Espace Disque et de Checkpoints sur Colab/Kaggle

- *Difficulté* : Les sessions d'entraînement sur Colab/Kaggle s'interrompaient brusquement en raison de manques d'espace disque ou de déconnexions, perdant tous les checkpoints.
- *Solution* : Implémentation d'une sauvegarde régulière des checkpoints vers Google Drive / Kaggle Dataset et ajout d'un script de reprise automatique de l'entraînement à partir du dernier checkpoint enregistré.

📦 E. Conflits de Dépendances Python

- *Difficulté* : Incompatibilités fréquentes de versions entre `transformers`, `peft`, `torch`, `torchao` et les environnements hôtes (Windows vs. Linux Colab). Certaines fonctions d'évaluation (comme `evaluation_strategy`) plantaient le code.

- *Solution* : Écriture de scripts d'installation robustes et écriture de wrappers adaptatifs selon la version de la bibliothèque détectée.
-

6. L'Évolution Speech-to-Speech : Analyse des Contraintes de Moshi (Kyutai)

Pour atteindre une réactivité ultime, l'intégration de modèles Speech-to-Speech natifs (comme **Moshi** développé par Kyutai) a été analysée. Bien que cette technologie élimine la cascade STT→LLM→TTS pour descendre sous les **200ms de latence**, elle impose 6 barrières techniques majeures :

1. La lourdeur de l'infrastructure matérielle (GPU requis)

- *Contrainte* : Moshi ne peut pas tourner sur un processeur classique (CPU). Le modèle complet (Helium 7B + Mimi) requiert au minimum **16 à 24 Go de mémoire vidéo (VRAM) dédiée et rapide**.
- *Impact* : Obligation de louer en production des serveurs cloud équipés de cartes professionnelles (NVIDIA A10G, A100 ou H100), représentant un coût mensuel fixe très élevé.

2. Le développement de la couche de transport audio (WebSockets / WebRTC)

- *Contrainte* : Moshi est livré sous forme de code d'inférence brut. Il ne fournit pas de solution client-serveur prête à l'emploi.
- *Impact* : Nécessité de développer de toutes pièces la couche de transport : capture du micro utilisateur, compression via le codec *Mimi*, envoi WebSocket, et gestion de la gigue réseau (Jitter Buffer) pour éviter les saccades audio.

3. La gestion complexe de la détection de parole (VAD) et des interruptions

- *Contrainte* : Pour qu'un dialogue soit réaliste, l'utilisateur doit pouvoir couper la parole à Moshi.
- *Impact* : Obligation de coder un système de purge de flux audio. Dès que l'utilisateur commence à parler, un signal d'interruption doit instantanément vider les buffers du serveur de génération pour que l'IA se taise sur le champ.

4. L'absence de support natif pour Windows

- *Contrainte* : Kyutai a optimisé Moshi exclusivement pour Linux et macOS.
- *Impact* : Pour le développement local sous Windows, il est obligatoire de passer par WSL2 (Windows Subsystem for Linux) et de configurer manuellement le partage des ressources GPU, ce qui ajoute de la complexité et des latences de traitement audio.

5. L'interfaçage avec le réseau téléphonique (VoIP / SIP)

- *Contrainte* : Moshi n'intègre aucune passerelle de télécommunication native.
- *Impact* : Si les apprenants doivent appeler l'IA depuis un téléphone ou un standard de centre d'appels, il faut développer une passerelle complexe SIP/WebRTC pour convertir

les flux RTC classiques vers le codec Mimi de Moshi.

6. L'intégration de la logique métier (Function Calling)

- *Contrainte* : Moshi génère un flux de parole brut sans capacité native d'interroger des bases de données.
 - *Impact* : Nécessité de développer un intercepteur de "monologue intérieur" (le texte généré par l'IA en parallèle de sa voix). Si l'intention d'effectuer une action (ex: bloquer une carte) est détectée, le système doit mettre en pause l'audio, interroger l'API bancaire de la BMCI, puis réinjecter le résultat sous forme de contexte textuel.
-

7. Bilan, État Actuel & Prochaines Étapes

État Actuel du Projet :

- Un **dataset bancaire BMCI** solide et complet.
- Un **modèle Qwen fine-tuné** (Qwen2.5-0.5B-BMCI-Client-Finetune-1) opérationnel localement.
- Une interface **Streamlit** fonctionnelle intégrant le chat, l'évaluation et des guardrails.
- Un **agent vocal en direct sur LiveKit** avec voix en colère stable (Marie) et résilience automatique (superviseur).

Prochaines Étapes d'Amélioration :

1. **Enrichissement du Dataset de Fine-Tuning** : Ajouter des exemples pour apprendre au modèle à résister aux pièges du conseiller, refuser les données confidentielles (sécurité), et ne pas inventer de faux détails bancaires.
2. **Optimisation des Checkpoints** : Augmenter le nombre d'époques d'entraînement sur Kaggle/Colab en stabilisant les exports automatiques de checkpoints.
3. **Étude de Faisabilité de Moshi / S2S** : Évaluer l'opportunité d'investir dans une infrastructure cloud GPU pour tester Moshi en environnement réel.